

LLMProvider

LLMProvider

Eine Schnittstelle, 43 Anbieter - mit integrierten Circuit Breakern, Wiederholungsversuchen und Gesundheitsprüfungen.

Zusammenfassung

LLMProvider ist ein generisches, wiederverwendbares Go-Modul, das eine einheitliche LLMProvider-Schnittstelle definiert sowie die dazugehörigen Resilienz-Muster für den Produktiveinsatz - Circuit Breaker, Gesundheitsüberwachung, Wiederholungsversuche mit Backoff, Lazy Loading - und 43 konkrete Provider-Implementierungen hinter diesem einen Vertrag bereitstellt, inklusive eines OpenAI-kompatiblen generischen Adapters und einer ehrlichen, ohne hartcodierte Fallbacks arbeitenden Modellerkennung.

Kurzbeschreibung

Ein wiederverwendbares Go-Modul, das eine einzige LLMProvider-Schnittstelle (Complete, CompleteStream, HealthCheck, GetCapabilities, ValidateConfig) sowie Fehlertoleranz-Mechanismen - Circuit Breaker, Gesundheitsmonitor, Wiederholungsversuche mit Jitter-Backoff, verzögerte Initialisierung - über 43 Provider-Adapter und einen generischen OpenAI-kompatiblen Adapter exponiert. Thread-sicher.

Ausführliche Beschreibung

LLMProvider ist die Abstraktionsschicht, die jedes LLM-nutzende System benötigt, die aber fast niemand richtig umsetzt - das unscheinbare Fundament, das eine Demo von einem System unterscheidet, das im echten Betrieb überlebt. Es definiert eine einzige, funktionsbewusste Schnittstelle - Complete, CompleteStream, HealthCheck, GetCapabilities, ValidateConfig -, sodass

Anwendungslogik stets denselben Vertrag adressiert, unabhängig davon, welcher der 43 Backends die Anfrage bedient. Gleichzeitig liefert es die operationelle Robustheit, die fragile Provider-Aufrufe in etwas verwandelt, das man bedenkenlos in Produktion einsetzen kann.

Ein dreistufiger Circuit Breaker (geschlossen → offen → halb-offen) umhüllt jeden Provider – *einschließlich seines Streaming-Kanals*, wobei ein leerer Stream korrekt als Fehler gewertet wird – und verhindert so, dass ein einzelnes fehlerhaftes Backend das gesamte System in Mitleidenschaft zieht. Ein zentraler CircuitBreakerManager überwacht alle Breaker gleichzeitig. Ein konfigurierbarer Gesundheitsmonitor durchläuft kontinuierlich die Zustände *gesund*, *degradiert*, *fehlerhaft* und *unbekannt* anhand von Schwellenwerten und Intervallen, sodass Verschlechterungen erkannt werden, bevor sie zu einem Ausfall führen.

Die Wiederholungslogik kombiniert exponentiellen Backoff mit Jitter und trifft zustandsabhängige Entscheidungen: Wiederholt werden nur Fehler, die es wert sind (429, 5xx, vorübergehende Netzwerkstörungen), während 4xx-Fehler oder abgebrochene Kontexte ignoriert werden. Die Verzögerungen sind begrenzt, um ein unkontrolliertes Aufschaukeln der Backoff-Zeiten zu verhindern. Das Lazy-Init-Muster verschiebt die Initialisierung jedes Providers auf den ersten tatsächlichen Aufruf – eine bewusste Designentscheidung, die die Registrierung aller 43 Provider praktisch kostenlos macht.

Das Modul liefert 43 konkrete Provider-Pakete sowie einen generischen OpenAI-kompatiblen Adapter, der die vollständige Schnittstelle für *jeden* /v1/chat/completions-Endpunkt implementiert – mit Bearer-Authentifizierung und korrekter SSE-Streaming-Verarbeitung inklusive [DONE]-Handling. So wird ein Anbieter ohne dediziertes Paket sofort zum gleichberechtigten Teilnehmer, sobald der Adapter auf dessen URL-Endpunkt gerichtet wird.

Anmeldedaten werden an genau einer Stelle aufgelöst (apikeys, unter strikter Einhaltung der ApiKey_<Provider>-Konvention), wodurch die gesamte Klasse von Fehlern ausgeschlossen wird, bei denen ein hartcodierter Schlüssel zwar die Testsuite besteht, der echte Schlüssel aber nie eingebunden wurde und das Produkt in Produktion versagt. Die Modellerkennung ist bewusst – fast schon stur – transparent: Sie fragt Live-APIs der Provider hinter einem TTL-Cache ab, und gemäß Governance-Richtlinien wurde die alte, hartcodierte Fallback-Ebene vollständig entfernt. Schlägt die Live-Erkennung fehl, gibt LLMProvider *nichts* zurück, statt einen veralteten Katalog auszuliefern. So erhält der Aufrufer niemals eine Modell-ID, die zwar gültig erscheint, sich aber nicht aufrufen lässt. Jeder Bestandteil ist thread-sicher für die parallele Nutzung ausgelegt.

Warum wir es entwickelt haben

Naive LLM-Aufrufe scheitern im Produktivbetrieb – Anbieter drosseln die Leistung, verschlechtern die Qualität oder fallen aus, und ein fehlerhafter Backend-Dienst kann das gesamte System mitreißen. Modellkataloge veralten, und hartcodierte Listen liefern Aufrufern IDs, die längst nicht mehr funktionieren. LLMProvider bündelt die Schnittstelle, die Resilienzmechanismen und eine zuverlässige Erkennung, sodass jeder Nutzer automatisch Fehlertoleranz und Korrektheit erhält – ohne zusätzlichen Aufwand.

Warum es ein Game-Changer ist

Es reduziert die Integration eines LLM-Anbieters auf einen einzigen Schritt: Implementiere eine Schnittstelle oder richte den generischen Adapter einfach auf einen Endpunkt aus – und schon wird dieser Anbieter automatisch und transparent mit Circuit Breaking, Gesundheitsüberwachung und verzögerten Wiederholungsversuchen mit Jitter umhüllt. Resilienz ist nicht länger etwas, das jedes Team aufs Neue (schlecht, unter Zeitdruck, nach dem ersten Ausfall) erfinden muss, sondern wird zum Standardverhalten der Bibliothek für alle 43 Backends. Die Zuverlässigkeitstechnik wird einmal entwickelt, gründlich getestet und von jedem übernommen, der sie importiert – kostenlos.

Was innovativ ist

- **Eine einzige fähigkeitsbewusste Schnittstelle** – Vervollständigung, Streaming, Gesundheitsstatus, Fähigkeiten und Konfigurationsvalidierung sind in einem einzigen Vertrag zusammengefasst, den jedes Backend identisch einhält.
- **Transparentes Circuit-Breaker-Wrapping - inklusive Streams.** Der Breaker schützt den Kanal von CompleteStream, nicht nur Anfrage/Antwort, und behandelt einen leeren Stream als das, was er wirklich ist: ein Fehler – mit deadlock-sicherer Benachrichtigung der Listener außerhalb des Sperrmechanismus.
- **43 Anbieterpakete + ein generischer OpenAI-kompatibler Adapter** – dedizierte Pakete bleiben schlank, und jeder nicht aufgelistete Anbieter, der /v1/chat/completions unterstützt, funktioniert sofort, sobald der Adapter darauf ausgerichtet wird.
- **Einheitliche Berechtigungsinstanz (apikeys)** – genau eine Stelle liest ApiKey_<Anbieter>-Umgebungsvariablen aus und beseitigt damit strukturell den Widerspruch „Tests laufen grün, Produkt ist kaputt“, statt nur davor zu warnen.

- **Ehrliche Modellerkennung (kein hartcodierter Fallback)**
– Live-Anbieter-APIs hinter einem TTL-Cache; bei einem Ausfall wird `nil` zurückgegeben, niemals ein veralteter oder erfundener Katalog, der nicht aufrufbare IDs ausgibt.
- **Lazy Init mit `sync.Once`** – Die Initialisierung wird auf den ersten Aufruf verschoben, sodass die Registrierung aller 43 Anbieter fast nichts kostet, bis tatsächlich einer genutzt wird.
- **Anti-Bluff, mehrsprachiger Challenge-Stack** – ein echter Runner, der Circuit Breaker, Gesundheitsstatus und Wiederholungsverhalten in fünf Regionen testet, gesteuert durch gepaarte Mutationstests (unmutierter Code muss mit Exit-Code 0 enden; eine injizierte Mutation muss Exit-Code 99 erzwingen). Ein bestandener Testlauf beweist damit nachweislich funktionierendes Verhalten.

Größte technische Herausforderungen & wie wir sie gelöst haben

- **Kaskadierende Anbieterausfälle.** Ein instabiles Backend darf nicht das gesamte System in den Abgrund reißen. Gelöst durch einen dreistufigen Circuit Breaker (geschlossen → offen → halb offen), der jeden Anbieter – *inklusive seines Streams* – transparent umhüllt, bei anhaltenden Fehlern öffnet, in halb offenem Zustand die Wiederherstellung prüft und zentral von einem `CircuitBreakerManager` koordiniert wird.
- **Vorübergehende Fehler und Ratenbegrenzungen.** Gelöst durch statusabhängiges exponentielles Backoff mit Jitter – $\min(\text{InitialDelay} \cdot \text{Multiplikator}^{(n-1)}, \text{MaxDelay}) \pm \text{Jitter}$ –, sodass Wiederholungsversuche sich verteilen, statt sich zu einer „Donnerherde“ zu synchronisieren. Es werden genau die Fehler wiederholt, die wiederholt werden sollten (429, 500, 502, 503, 504 und Netzwerkfehler), und es werden keine Versuche an einen abgebrochenen Kontext oder andere 4xx-Fehler verschwendet.
- **Skalierung auf viele registrierte, aber ungenutzte Anbieter.** Bei 43 registrierten Anbietern, von denen in einem bestimmten Dienst nur wenige aktiv sind, wäre eine sofortige Initialisierung reine Verschwendung. Gelöst durch verzögerte Initialisierung, geschützt durch `sync.Once`, sodass nur die tatsächlich aufgerufenen Anbieter ihren Setup-Aufwand verursachen.
- **Ausgabe ungültiger Modell-IDs.** Gelöst durch die vollständige Entfernung der hartcodierten Erkennungs-Fallback-Ebene (gemäß CONST-036) und die Rückgabe von `nil` bei einem Live-Erkennungsfehler – zusätzlich mit einer defensiven Kopie bei der Rückgabe, sodass ein Aufrufer den Cache nicht mutieren oder mit einem anderen Leser in einen Wettlauf geraten kann. Korrektheit wird strukturell erzwungen, nicht durch Konvention.

- **Streaming + Nebenläufigkeitskorrektheit.** Das subtile Problem ist ein Deadlock zwischen dem Sperrmechanismus des Breakers und seinen Listener-Callbacks. Gelöst durch das Erstellen von Snapshots der Listener und deren Benachrichtigung außerhalb des Sperrmechanismus mit einem 5-Sekunden-Timeout sowie durch das Entsperren vor der Benachrichtigung bei einem Reset – wobei jede Komponente für nebenläufige Nutzung ausgelegt und durch die -race-Suite abgesichert ist.

Technologie-Stack

- **Go (1.25.3)** — ausgewählt für erstklassige Nebenläufigkeit, statische Binärdateien und eine robuste Standardbibliothek; umfasst das Modul, die Schnittstelle, alle Resilienz-Primitiven und sämtliche 43 Adapter.
- **net/http (Standardbibliothek)** — absichtlich abhängigkeitsfreie HTTP-Implementierung: treibt die Provider-spezifischen Clients, den generischen OpenAI-kompatiblen Adapter sowie die Live-Discovery-Aufrufe an, sodass kein Drittanbieter-Transport auditiert oder gepatcht werden muss.
- **logrus** — strukturiertes, ebenenbasiertes Logging genau dort, wo Betreiber:innen Transparenz benötigen: innerhalb der Zustandsübergänge des Circuit Breakers und im Discovery-Pfad.
- **testify** — steuert die Testsuite und vor allem das Mutations-Branch-Pinning, das einem erfolgreichen Durchlauf Bedeutung verleiht.
- **yaml.v3** — parst die i18n-Bundles und Konfigurationen in einem Format, das für Menschen bearbeitbar bleibt.
- **digital.vasic.models** — die gemeinsamen Typen LLMRequest / LLMResponse / ProviderCapabilities, an einem zentralen Ort gehalten, damit jeder Adapter dieselbe „Sprache“ spricht (eine dokumentierte Laufzeitabhängigkeit).
- **Eigenentwickelte Pakete** — circuit, health, retry, apikeys, discovery, providers/ (43 Anbieter + generic) und i18n: Die Resilienz- und Integrationsschicht ist in kleine, unabhängig testbare Einheiten unterteilt, statt in einem Monolithen zusammengefasst.
- **.env + ~/api_keys.sh (ApiKey_<Provider>-Konvention)** — eine einzige, eindeutige Quelle für Anmeldedaten, sodass Schlüssel in Tests und Produktion auf dieselbe Weise eingebunden werden.
- **Makefile-Race-Suite (-race -p 1) + Challenge-Runner** — das Anti-Bluff-Rückgrat: Der Race-Detektor beweist die Korrektheit der Nebenläufigkeit, und der Challenge-Runner testet reales Verhalten unter Chaos, DDoS, Skalierung, Stress, Live-Discovery und Szenarien ohne Unterbrechung.

Status & Transparenzhinweise

- **Status: Beta.** Ein entkoppeltes, wiederverwendbares Modul; das GitHub-Repository ist öffentlich.
- **Lizenz: Unklar.** Inkonsistent — doc.go gibt MIT an, während eine LICENSE-Datei im Apache-2.0-Stil vorhanden ist — vor der Veröffentlichung prüfen.
- LLMsVerifier ist die übergeordnete Quelle der Wahrheit für den kanonischen Modellkatalog. Die helix-deps.yaml-Manifestdatei wirkt veraltet (deklariert deps: [], obwohl die Dokumentation eine Abhängigkeit von digital.vasic.models angibt); die „Tier 2 (models.dev)“-Discovery ist ein geplanter Stub und derzeit nicht aktiv.

Prioritätsstufe: Helix-primär (LLM-Infrastruktur-Cluster — entkoppeltes, wiederverwendbares Modul). Rangiert nach HelixTrack.